SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I G. Ram Srinivas *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		documentation			

DEBUGGING & OPTIMISATION TASK — ANSWERS

Course: Cryptography and Network Security (CSA51)

Note: The questions describe faulty Python implementations but do not provide the actual source code. Therefore, the answers below identify common root causes and give corrected reference patterns rather than claiming to patch unseen code.

1. Debugging Hash Function Implementation (SHA Family)

Likely errors and root causes

A SHA implementation can produce inconsistent digests when padding is added incorrectly, the message length is recorded in the wrong units, blocks are processed at the wrong size, or the chaining state is not reset/updated correctly. SHA-256 processes 512-bit (64-byte) blocks and appends a 64-bit big-endian representation of the original message length in bits. The padding begins with one '1' bit (0x80), followed by zero bits until the message is 56 bytes modulo 64, then the 8-byte length field. Bitwise operations must be masked to 32 bits, and word parsing/output must use big-endian order.

Recommended correction

Use a well-tested standard library implementation instead of rewriting SHA internals. Stream large files in chunks so memory usage stays bounded. The following pattern is correct for SHA-256 and handles files larger than 50 MB:

```
import hashlib

def sha256_file(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()

# Example:
# print(sha256_file("large_file.bin"))
```

Debugging checklist

1. Test empty input, a one-byte input, inputs near the 55/56/63/64-byte padding boundaries, and multi-block inputs. 2. Compare output with hashlib.sha256 for the same byte sequence. 3. Confirm the original length is measured in bits, not bytes. 4. Confirm 32-bit additions and rotations are implemented exactly as specified. 5. Confirm input is read in binary mode and chunks are passed to one persistent digest object.

Integrity analysis

A checksum such as a simple sum or CRC is useful for accidental transmission errors, but it is not designed to resist deliberate manipulation. SHA-256 makes it computationally infeasible to find a different file with the same digest under current practical assumptions. For authenticity against an attacker who can replace both file and digest, use a keyed MAC (such as HMAC-SHA-256) or a digital signature; an unkeyed hash alone does not prove who created the file.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

Likely faults

Handshake failures commonly result from expired or not-yet-valid certificates, hostname mismatch, an untrusted issuer, missing intermediate certificates, incompatible protocol/cipher settings, incorrect key-share handling, or inconsistent client/server state. Disabling certificate verification is not a valid fix because it permits man-in-the-middle attacks.

Debugging and secure corrections

Validate the certificate chain against a trusted CA store, check validity dates and the requested hostname, and preserve normal TLS verification. Log the specific failure reason without logging secrets. Use a maintained TLS library and supported TLS 1.2/1.3 settings rather than implementing cryptographic handshake messages manually. Ensure both peers agree on the negotiated version, cipher suite, and key-exchange parameters.

Latency improvements

Use TLS 1.3 where supported, reduce unnecessary round trips, reuse established connections where practical, and use session resumption (session tickets or PSK-based resumption) through the TLS library. Cache only valid resumption state, set sensible expiration, and invalidate it when policy or credentials require. Session resumption reduces repeated handshake work but must not be implemented by reusing a static traffic key across unrelated connections.

```
# Example: secure client context in Python
import ssl
import socket

context = ssl.create_default_context()
# Keep certificate and hostname verification enabled.
# The context can be reused for connections; the TLS library
# manages protocol details and supported session behavior.

with socket.create_connection(("example.com", 443), timeout=5) as sock:
    with context.wrap_socket(sock, server_hostname="example.com") as tls:
        print(tls.version())
        print(tls.cipher())
```

Performance versus security

Connection reuse and session resumption can reduce CPU cost and handshake latency. The trade-offs include the need to protect ticket keys, limit ticket lifetime, and consider replay risks for early data (0-RTT). Do not accept 0-RTT for non-idempotent actions unless the application is designed to handle replay. Keep strong cipher suites, certificate checks, and current protocol versions; never trade away authentication for speed.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Likely errors

Invalid DSA signatures can result from invalid domain parameters (p, q, g), using a private key outside the required range, generating a nonce k that is zero/out of range or reused, incorrect modular inverses, hashing inconsistently, or verifying with the wrong public key. DSA requires q to divide p–1, 1 < g < p, and a private key x in the range 1 ≤ x ≤ q–1. The per-message nonce k must be secret, uniformly generated, and never reused.

Signature equations

For a message hash H(m), private key x and fresh nonce k, compute r = (g^k mod p) mod q and s = k■¹(H(m) + xr) mod q. Reject the signature if r = 0 or s = 0. Verification computes w = s■¹ mod q, u1 = H(m)w mod q, u2 = rw mod q, and v = ((g^u1 · y^u2 mod p) mod q), where y = g^x mod p. The signature is valid only if 0 < r,s < q and v = r.

Safe implementation approach

Use a maintained cryptographic library and standardized parameter/key generation rather than custom DSA code. For new systems, follow current policy and prefer modern signature schemes supported by the platform (for example, Ed25519 or ECDSA with appropriate curves); DSA is deprecated or restricted in many modern environments.

```
# Reference checks for modular arithmetic (illustrative only)
def valid_dsa_signature_range(r, s, q):
    return 0 < r < q and 0 < s < q

# Python's pow(a, -1, modulus) computes a modular inverse
# when the inverse exists (Python 3.8+):
# k_inv = pow(k, -1, q)
```

Optimization and randomness security

Use fast modular exponentiation (pow(base, exponent, modulus)), which avoids constructing huge intermediate powers. Validate parameters once during key setup rather than on every signature. Use a cryptographically secure random generator or a library's deterministic nonce construction where standardized and supported. Reusing or leaking k can reveal the private key x; weak randomness can therefore compromise every signature made with that key. Never use Python's random module for cryptographic nonces.

4. Debugging Key Management System

Storage and lifecycle flaws

Storing keys in plaintext files, source code, logs, or environment dumps exposes them to anyone who gains access to the host or repository. Weak encryption, hard-coded keys, shared keys for unrelated purposes, excessive permissions, and missing backups or rotation procedures also create serious risks. If an attacker obtains a private key or encryption key, they may decrypt protected data, impersonate users or services, and forge trusted messages.

Corrected design

Generate keys with a cryptographically secure random source or a vetted key-management service. Store production keys in a KMS/HSM or operating-system secret store; if local encrypted storage is unavoidable, use authenticated encryption with a separately protected key-encryption key. Apply least-privilege access control, authenticate and authorize every key operation, audit use, and never print secret material. Separate keys by purpose and environment.

Lifecycle controls

Generation: use approved algorithms and key sizes. Distribution: use authenticated secure channels or a KMS; do not email raw keys. Storage: encrypt and restrict access. Usage: expose cryptographic operations through controlled APIs where possible. Rotation: define schedules and rotate immediately after suspected compromise. Revocation and destruction: disable compromised/expired keys and securely remove retired copies, including backups where feasible. Recovery: maintain protected backups and test restoration.

```
# Example: authenticated encryption for a small secret
# Install: pip install cryptography
from cryptography.fernet import Fernet

key = Fernet.generate_key()       # Protect this key separately!
fernet = Fernet(key)
ciphertext = fernet.encrypt(b"example secret")
plaintext = fernet.decrypt(ciphertext)

# Production: keep `key` in a KMS/secret manager,
# not beside the ciphertext or in source code.
```

Security analysis

Cryptography is only as strong as its key management. A strong cipher cannot protect data if its key is stolen, reused improperly, or left unprotected. Good lifecycle management limits who can access keys, reduces the time a compromised key remains useful, and supports accountability through auditing.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

Likely causes of corruption and slowness

Data corruption may arise from incorrect chunk boundaries, partial reads/writes, packet reordering or loss that is not handled, mismatched encryption nonces/counters, incorrect authentication-tag verification, or writing chunks in the wrong order. Slow transfers may be caused by tiny chunks, repeated connection setup, single-threaded bottlenecks, unnecessary copying, or encryption implemented inefficiently.

Secure debugging approach

Use an established SFTP implementation over SSH rather than designing a new transfer protocol. Ensure the receiver writes all bytes returned by each read, handles partial writes, and verifies file size and a trusted digest after transfer. If using authenticated encryption in a custom simulation, every chunk must have unique nonces and its authentication tag must be verified before the plaintext is accepted. Never reuse a nonce with the same key, and never continue after tag verification fails.

Throughput improvements for files ≥100 MB

Stream data in reasonably large chunks (for example, 256 KiB to 1 MiB), avoid loading the entire file into memory, reuse the SSH connection, and tune buffer sizes based on measurement. Use efficient library implementations and profile CPU, disk, and network throughput before adding concurrency. For very large files, resume support can avoid restarting from byte zero after an interruption; authenticate the resumed session and verify the final file.

```
# Simple streaming pattern for local hashing/verification
import hashlib

def sha256_path(path, chunk_size=1024 * 1024):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(chunk_size), b""):
            h.update(chunk)
    return h.hexdigest()

# Compare sender/receiver digests through an authenticated channel.
# For actual transfers, prefer a maintained SFTP/SSH library.
```

Comparison with insecure transfer

Plain FTP or a custom unauthenticated transfer can expose credentials and file contents, and an attacker may modify data in transit. SFTP protects confidentiality and integrity in transit and authenticates the server through SSH host-key verification; user authentication and permissions protect access. An end-to-end SHA-256 digest can detect mismatch when the expected digest is trusted, while SSH/SFTP already protects the transport. Do not disable host-key checking merely to avoid connection errors.

Conclusion

The best optimization is to measure the bottleneck, stream data efficiently, reuse secure connections where appropriate, and rely on proven protocols and libraries. Security checks—host authentication, encryption, and integrity verification—must remain enabled.